Cascade risk analyzer based on bootstrap percolation: Methodology

Alvaro Monclús

March 2026

BACHELOR'S THESIS

Instituto Superior de Engenharia do Porto

1 Introduction

Modern complex systems such as social networks, financial systems, infrastructure and communication networks are often vulnerable to cascade phenomena, where a local and seemingly insignificant event may trigger a systematic failure or degradation of the network. Understanding and predicting these cascades is therefore a fundamental problem in network design.

This project proposes the development of a cascade risk analyzer, a computational tool that evaluates how susceptible to cascade phenomena a network is using algorithmic models based on bootstrap percolation. The system will allow users to analyze how local activation rules can lead to global effects in their particular network and determine the weakest nodes that contribute to this effect.

The tool will combine simulation algorithms, network analysis and visualization to estimate cascade risks and identify the structural vulnerabilities of the network.

2 Classic and bootstrap percolation

2.1 Classic percolation

Before jumping into the product and methodology, we need to establish an overview of the theoretical aspects of percolation.

Imagine a large chessboard like those found in parks in a fall afternoon. You notice that some squares are full of fallen leaves while others remain clean in a seemingly random pattern. To get across from one side of the chessboard to the other you decide to jump from clean square to clean square, while avoiding the ones full of leaves. As it is fall and the ground is full of leaves you cannot get from one side to the other without having to step on the covered squares, so you decide to come back in the spring, when fallen leaves are in less quantity. Now, as less squares are covered, you can cross from one side to the other with ease. This is nearly the problem of percolation. [1]

The above example illustrates that percolation is not only the problem of knowing how many squares are covered but the spacial connectivity of them and the main question of *How many leaves can we have at most so that we can traverse the chess board?*

Given that leaves are distributed randomly we can define a probability of occupation p and ask a new question: Is there a threshold value p (we will call it p_c) for which a giant cluster that lets us cross from one side to the other is formed?

2.1.1 Formalization

To formalize this idea, percolation is typically defined on a graph or lattice $G = (V, E)$ where V represents the set of sites (or nodes) and E the connections between them. In *site percolation*, each site $v \in V$ is independently occupied with probability p and empty with probability $1 - p$. In *bond percolation*, instead of nodes, each edge $e \in E$ is independently active with probability p and inactive otherwise.

A *cluster* is defined as a maximal connected component of occupied sites (or active edges). The central object of study in percolation theory is the emergence of a *giant cluster*, a

connected component whose size scales with the size of the system.

As the occupation probability p increases, the structure of the clusters changes. There exists a critical probability p_c, known as the *percolation threshold*, such that for $p < p_c$ all clusters remain small, while for $p > p_c$ a giant cluster appears with high probability, allowing connectivity across the system.

2.2 Bootstrap percolation

This study will mainly focus on bootstrap percolation, a percolation process mainly applicable to studying the cascade effect in networks. k-bootstrap percolation is defined as follows. Given a graph G and a set $A_0 \subseteq V(G)$ of initially infected nodes and an infection parameter $k \in \mathbb{N}$, a non-infected node becomes infected if it has at least k infected neighbors and stays in that state forever. The process reaches an end when either there are no more healthy nodes or all remaining healthy nodes have less than k infected neighbors.[2]

This process evolves in discrete time steps. Let A_t denote the set of infected nodes at time t, where A_0 is the initial configuration. At each step, nodes that have at least k infected neighbors in A_t become infected, producing a new set A_{t+1} with $A_t \subseteq A_{t+1}$. A central question in bootstrap percolation is determining whether the infection spreads to the entire graph, that is, whether there exists a time t such that $A_t = V(G)$. This property is known as *complete percolation*.

3 Product overview

The main objective of the product is to provide a simulation and analysis environment for users to test the robustness of their networks in relation to cascade effects.

More specifically, the tool will allow the user to:

- Generate or load networks
- Define cascade parameters based on bootstrap percolation rules
- Simulate the cascade process
- Analyze the cascade behavior
- Identify structures vulnerable to cascade effects

4 Core functionality

The risk analyzer will provide the following functionalities.

4.1 Network generation and input

The system will support the creation of different artificial graph models commonly studied in network science, including:

- Erdős–Rényi random graphs
- Random geometric graphs

- Regular lattice graphs

These graph families were selected because they exhibit different structural properties such as randomness, spatial locality, and regular connectivity, which strongly influence cascade dynamics.

Users will be able to control important parameters such as:

- Number of nodes
- Connection probability or connection radius
- Graph density

Additionally, the user will be given the option to input their own graph following the DIMACS format described below, where a c indicates a comment line, p states the problem line as `p edge <number_of_vertices> <number_of_edges>` and e the edge line, where `<e v u>` indicates an edge between vertices v and u.

Listing 1: Example of a graph in DIMACS format

```
c Example graph
c 5 nodes, 6 edges
p edge 5 6
e 1 2
e 1 3
e 2 4
e 3 4
e 4 5
e 2 5
```

4.2 Cascade simulation

Cascade dynamics will be simulated using a k-bootstrap percolation model.

The simulation process will follow these steps:

1. A graph is generated or loaded.
2. An initial set of active nodes is selected randomly according to an activation probability.
3. During each iteration, inactive nodes become active if at least k of their neighbors are active.
4. The process continues until no further activations occur.

The simulation terminates once the system reaches a stable configuration.

4.3 Cascade metrics and risk indicators

To evaluate cascade behavior, the tool will compute several metrics, including:

- Final cascade size (fraction of active nodes)
- Size of the largest active component

- Number of activation waves
- Critical seed size (Number of initially active nodes to trigger a cascade)
- Percolation threshold
- Time required for stabilization
- Probability of global cascade

These metrics allow the system to quantify how likely a local activation event is to propagate through the entire network. Combining these metrics, a final robustness score will be decided giving a weight to each metric.

4.4 Visualization and analysis

The system will include visualization tools to help interpret cascade behavior. Possible visual outputs include:

- Network activation animations
- Cascade evolution over time
- Plots showing cascade size as a function of the initial activation probability
- Phase transition curves

These visualizations will allow users to observe how cascade dynamics emerge from local interaction rules and how network structure influences cascade propagation.

5 Technical implementation

The risk analyzer will be implemented as a software system consisting of several interconnected components. The main components are described below.

5.1 Simulation engine

The simulation engine is responsible for:

- Graph generation
- Cascade simulation based on k-bootstrap percolation

Languages and libraries:

- **Python** as the primary programming language
- **NetworkX** for graph generation and network representation
- **NumPy** and **SciPy** for efficient numerical computations

5.2 Analysis module

The analysis module computes cascade metrics and performs statistical evaluations.

- Calculation of final cascade size, largest active component, activation waves, and probability of global cascades
- Monte Carlo simulation framework to estimate statistical properties

Languages and libraries:

- **Python** with **Pandas** for data handling
- **Matplotlib** and **Seaborn** for plotting and visualization of metrics

5.3 Visualization module

The visualization module provides interactive and static visualizations of cascade processes, allowing users to explore the dynamics of network activation.

- Animated visualization of cascade propagation on the network
- Phase transition and cascade evolution plots

Languages and software:

- **Python** with **Matplotlib**, **Seaborn**, and optionally **Plotly** for interactive visualizations
- **Streamlit** or **Dash** for creating a simple graphical user interface

5.4 System design considerations

The system will be designed with the following principles:

- Modularity: each component (simulation, analysis, visualization) can operate independently
- Reproducibility: results of simulations can be saved and reloaded
- Scalability: the system should handle graphs of varying sizes efficiently

Additionally, if time and complexity allows it, the risk analyzer will be deployed as a web service using **Docker** for containerization and using **Streamlit** or **Dash** for the interactive interface with the potential integration of a lightweight backend server like **FastAPI** to handle simulations and user requests. Again, this final part would only be as an added bonus if our timeline allows it.

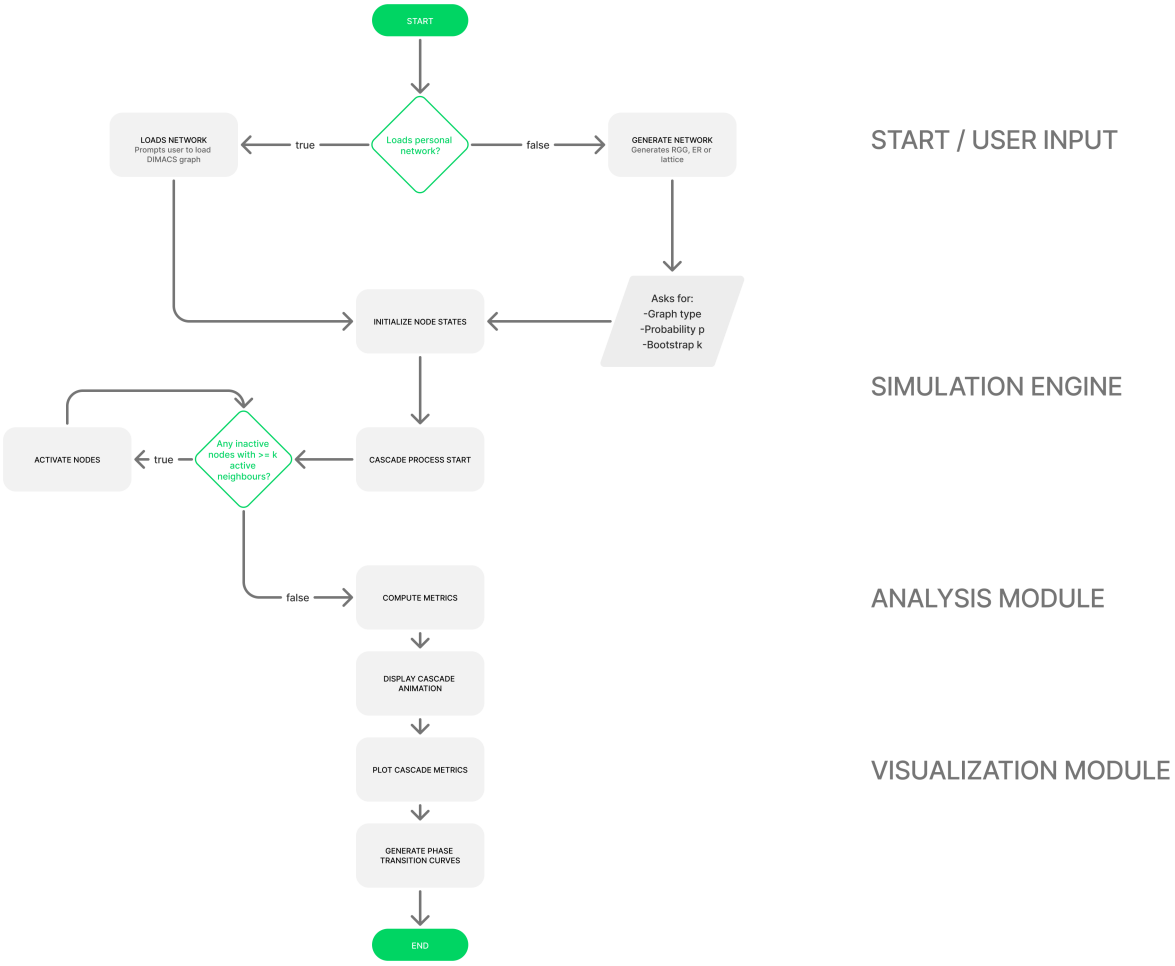

Figure 1: Flow diagram of the system

References

[1] Rudolf A. Roemer. Percolation, Renormalization and the Quantum-Hall Transition. Institut für Physik, Technische Universität, Chemnitz, 2001.

[2] Kang, Mihyun and Missethan, Michael and Schmid, Dominik. (2024). Bootstrap percolation on the high-dimensional Hamming graph. 10.48550/arXiv.2406.13341.